

# Dry Bean Dataset 多分类机器学习项目报告

## 摘要

本项目基于 Dry Bean Dataset (干豆数据集), 完成了一个完整的机器学习工程流程: 数据清洗与特征工程、四种多分类算法的实现与对比 (SVM、随机森林、XGBoost、多层感知机)、系统化的模型评估 (精度、鲁棒性、推理速度、过拟合分析), 并构建了模块化的命令行工具系统。数据集经过人工“污染”处理 (标签错误、缺失值、格式异常), 模拟了真实场景下的数据质量问题。最终实验表明, SVM 在测试精度上表现最优 (93.42%), MLP 在推理速度上最快 (0.0006 ms/样本), XGBoost 在精度与速度之间取得了良好平衡。项目代码已上传至 GitHub, 提供完整的 README 和使用说明。

## 1. 引言

### 1.1 问题背景

干豆 (Dry Bean) 是重要的农作物之一, 不同品种的干豆在外观上有细微差异。通过计算机视觉技术自动分类干豆品种, 对于农业自动化、质量检测等领域具有重要意义。本数据集包含从干豆图像中提取的 16 个几何形状特征, 任务是将每个样本准确分类到 7 个品种之一。

### 1.2 项目目标

根据课程期末考核要求, 本项目需完成以下目标:

- 1. **数据分析 (5%)**: 对输入数据进行主观观察和判断, 识别数据污染情况
- 2. **数据处理 (30%)**: 完成数据清洗和特征工程
- 3. **多算法实验分析 (30%)**: 实现至少 3 种多分类算法 (含至少 1 种课堂未讲授算法), 对比测试精度、Loss 曲线、推理速度、鲁棒性
- 4. **系统展示 (30%)**: 模块化工程架构, 命令行统一调用, 上传 GitHub 并完善 README
- 5. **课程总结 (5%)**: 总结课程收获, 提出评价与建议

## 2. 数据集描述

### 2.1 基本信息

| 属性   | 值               |
|------|-----------------|
| 特征数量 | 16 (全部为数值型几何特征) |
| 目标类别 | 7 种干豆品种         |
| 训练集  | 9,527 样本 (原始)   |
| 测试集  | 2,737 样本 (原始)   |
| 验证集  | 1,347 样本 (原始)   |
| 总样本数 | 13,611          |

### 2.2 特征列表

所有特征均为从干豆图像中提取的几何形状描述子:

| 特征              | 含义   | 特征             | 含义       |
|-----------------|------|----------------|----------|
| Area            | 面积   | EquivDiameter  | 等效直径     |
| Perimeter       | 周长   | Extent         | 外接矩形面积比  |
| MajorAxisLength | 长轴长度 | Solidity       | 凸包面积比    |
| MinorAxisLength | 短轴长度 | roundness      | 圆度       |
| AspectRatio     | 长宽比  | Compactness    | 紧密度      |
| Eccentricity    | 偏心率  | ShapeFactor1-4 | 形状因子 1-4 |
| ConvexArea      | 凸包面积 |                |          |

## 2.3 类别分布

| 类别       | 样本数   | 占比    |
|----------|-------|-------|
| DERMASON | 3,546 | 26.1% |
| SIRA     | 2,636 | 19.4% |
| SEKER    | 2,027 | 14.9% |
| HOROZ    | 1,928 | 14.2% |
| CALI     | 1,630 | 12.0% |
| BARBUNYA | 1,322 | 9.7%  |
| BOMBAY   | 522   | 3.8%  |

数据集呈现不均衡分布, DERMASON 类别占比最高 (26.1%), BOMBAY 类别最少 (仅 3.8%)。这种类别不平衡要求在评估时同时使用 weighted 和 macro 两种平均方式来综合衡量模型性能。

## 3. 数据分析 (评分占比 5%)

本节针对输入数据做主观观察和判断说明, 识别数据污染情况。

### 3.1 数据污染识别

数据集被人工“污染”以模拟真实场景中的数据质量问题, 经系统检测共发现以下四类污染:

#### (1) 类别标签错误

数据集中存在多种标签变体, 原始数据出现了 25 种不同的类标签 (实际应仅有 7 种):

| 污染类型          | 示例                             | 正确标签                           | 影响样本数 |
|---------------|--------------------------------|--------------------------------|-------|
| Leet-speak 替换 | D3RMAS0N, H0R0Z, S3K3R, B0MBAY | DERMASON, HOROZ, SEKER, BOMBAY | 171   |
| 大小写变体         | dermason, horoz, cali          | DERMASON, HOROZ, CALI          | ~200  |

| 污染类型   | 示例                     | 正确标签              | 影响样本数 |
|--------|------------------------|-------------------|-------|
| 尾部多余空格 | “BAR-BUNYA”,<br>“SIRA” | BARBUNYA,<br>SIRA | ~200  |

Leet-speak 是一种用数字替换字母的书写风格 (3→E, 0→O, 4→A), 在数据采集或录入过程中被引入。

## (2) 缺失值

- Perimeter 列: 677 个 NaN 缺失值 (占总体的 4.97%)
- Solidity 列: 388 个缺失值, 其中 270 个以“?”字符标记, 118 个为 NaN

## (3) 格式异常

- Compactness 列: 388 个值带有“cm”后缀 (如“0.8194 cm”), 导致该列被 pandas 读为字符串类型, 无法直接参与数值运算
- Solidity 列: 270 个值为“?”字符串

## (4) 重复样本

共发现 49 个完全重复的行 (所有特征值完全一致)。

## 3.2 异常值分析

使用 3 倍 IQR (四分位距) 方法检测异常值:

| 特征              | 异常值数量 | 占比    | 备注                         |
|-----------------|-------|-------|----------------------------|
| Area            | 518   | 3.81% | 含物理不可能的负值 (min = -203,536) |
| ConvexArea      | 500   | 3.67% |                            |
| MinorAxisLength | 499   | 3.67% |                            |
| EquivDiameter   | 229   | 1.68% |                            |
| ShapeFactor4    | 209   | 1.54% |                            |
| Solidity        | 206   | 1.51% |                            |

特别值得注意的是 Area 列出现了负值 (最小值 -203,536), 在物理意义上面积不可能为负, 这表明存在数据录入或传感器错误。

## 3.3 数据可视化与观察

以下通过 4 张图表对数据特征进行可视化分析 (完整大图见 output/figures/ 目录):

图 1: 类别分布柱状图

从图中直观可见类别不均衡现象: DERMASON 类别一家独大, 而 BOMBAY 样本量仅为其 1/7。这提示在模型评估时, 单纯的 accuracy 可能具有误导性——即使模型将所有样本预测为 DERMASON, 也能获得 26% 的准确率。因此必须关注 Macro F1 等不受类别分布影响的指标。

图 2：特征箱线图

箱线图揭示了两个关键问题：一是各特征的量纲差异极大——Area 的量级在  $10^{4-105}$ ，而 ShapeFactor1 仅在  $10^{-3}$  量级，相差 7 个数量级。这直接证明了标准化的必要性，否则基于距离的算法（SVM、MLP）将被大尺度特征主导。二是多个特征存在显著的离群点，验证了前文异常值检测的结果。

图 3：特征相关性热力图

热力图揭示了特征间的高度共线性，这是从干豆图像中提取几何特征的自然结果：- Area 与 ConvexArea 相关系数接近 1.00（凸包面积天然接近实际面积）- Area 与 EquivDiameter 相关系数 0.99（等效直径由面积公式导出）- Perimeter 与 MajorAxisLength 相关系数 0.98 - ShapeFactor2 与 Compactness 呈强负相关 (-0.90)

这些高相关性提示特征之间存在显著冗余，为后续可选的 PCA 降维提供了依据。

图 4：按类别的特征分布直方图

从 Area、Perimeter 等关键特征的类条件分布来看：- BOMBAY 类在多个特征上与其他类有较好的可分性（分布中心明显偏离）- SEKER 和 SIRA 在多数特征上分布几乎重叠，难以区分 - DERMASON 和 HOROZ 之间存在较大的分布重叠区域

这解释了为何后续混淆矩阵中 SEKER-SIRA 和 DERMASON-SIRA 之间容易出现误分类。

## 4. 数据处理（评分占比 30%）

本节针对输入数据做数据清洗和特征工程，说明每步处理前后的变化。

### 4.1 数据清洗流程

#### 步骤 1：标签标准化

建立映射表处理三类标签污染：- Leet-speak 映射：{D3RMASON→DERMASON, HOROZ→HOROZ, S3K3R→SEKER, BOMBAY→BOMBAY} - 大小写统一：所有标签转为大写 - 空格清除：.strip() 去除首尾空格

标准化后，类别标签从 25 种恢复为正确的 7 种，共纠正 171 个 Leet-speak 标签 + 约 400 个大小写/空格变体标签。

#### 步骤 2：特征列类型修复

- Solidity 列：将“?”替换为 NaN，整列转为 float64
- Compactness 列：正则去除“cm”后缀，转为 float64
- 所有 16 个特征列统一转换为数值类型

#### 步骤 3：缺失值处理

删除含有 NaN 的行。选择删除而非填充的理由：- 缺失值占总样本比例较小（Perimeter ~5%，Solidity ~2.8%）- 缺失样本在三个数据集中均匀分布 - 几何特征之间高度相关但并非线性关系，简单插值可能引入偏差

#### 步骤 4：去重

删除 49 个完全重复的样本行。

#### 步骤 5：异常值裁剪 (Clipping)

使用  $3 \times \text{IQR}$  边界对所有特征进行裁剪，将超出  $[\text{Q1}-3 \times \text{IQR}, \text{Q3}+3 \times \text{IQR}]$  的值替换为边界值。选择裁剪而非删除的理由：保留了样本的其他有效特征信息，同时抑制了极端值对模型训练的影响。

### 4.2 清洗效果统计

| 数据集 | 原始行数   | 清洗后    | 移除    | 移除率   |
|-----|--------|--------|-------|-------|
| 训练集 | 9,527  | 8,581  | 946   | 9.93% |
| 测试集 | 2,737  | 2,477  | 260   | 9.50% |
| 验证集 | 1,347  | 1,226  | 121   | 8.98% |
| 合计  | 13,611 | 12,284 | 1,327 | 9.75% |

数据保留率 90.25%，在有效清洗的同时保留了绝大部分样本。三个数据集移除比例接近 (9-10%)，说明污染是均匀施加的。

### 4.3 特征工程

#### (1) Z-score 标准化

采用 `StandardScaler` 将每个特征转换为均值为 0、标准差为 1 的分布。

选择 Z-score 而非 Min-Max 归一化的理由：- Z-score 对异常值相对鲁棒（已通过裁剪处理）- 保留特征的相对距离信息 - SVM (RBF 核) 和 MLP 对特征尺度敏感，标准化是必要前提 - 树模型 (RF、XGBoost) 不受影响但也不受影响

#### (2) 标准化效果验证

标准化前：Area (均值 52,782) 与 ShapeFactor1 (均值 0.0066) 量级差距约  $10^7$ 。标准化后：所有 16 个特征均值为 0、标准差为 1，消除了量纲差异。

#### (3) PCA 降维 (可选功能)

系统通过 `--pca` 参数支持可选的 PCA 降维，默认保留 95% 方差。鉴于特征间高度相关 (Area 与 ConvexArea 相关系数  $\sim 1.0$ )，PCA 可有效去冗余。但默认关闭以保留特征的可解释性。

## 5. 多算法实验分析 (评分占比 30%)

本节实现至少 3 种多分类算法 (含至少 1 种课堂未讲授算法)，从测试精度、Loss 曲线、推理速度、鲁棒性四个维度进行对比。

### 5.1 算法选择

| 算法               | 类型                      | 课堂讲授?    | 关键超参数                                           |
|------------------|-------------------------|----------|-------------------------------------------------|
| SVM (RBF Kernel) | 支持向量机                   | 是        | C=10, kernel=rbf, gamma=scale                   |
| Random Forest    | 集成学习 (Bagging)          | 是        | n_estimators=200, max_depth=15                  |
| <b>XGBoost</b>   | <b>梯度提升树 (Boosting)</b> | 否 (课外自学) | n_estimators=200, lr=0.1, max_depth=6           |
| MLP              | 多层感知机 (神经网络)            | 部分涉及     | hidden=(256,128,64), ReLU, Adam, early_stopping |

**课外算法说明：** XGBoost (eXtreme Gradient Boosting) 是陈天奇于 2016 年提出的梯度提升决策树工业级实现。相比传统 GBDT, XGBoost 引入了二阶泰勒展开近似目标函数、正则化项控制模型复杂度、列采样、并行化特征计算等创新。它在 Kaggle 竞赛中长期占据统治地位, 是工业界应用最广泛的机器学习算法之一。本项目通过查阅官方文档和学术论文实现了 XGBoost 的完整训练与评估流程。

## 5.2 测试集精度对比

| 算法            | Accuracy      | Precision (Macro) | Recall (Macro) | F1 (Macro)    | F1 (Weighted) |
|---------------|---------------|-------------------|----------------|---------------|---------------|
| <b>SVM</b>    | <b>0.9342</b> | <b>0.9447</b>     | <b>0.9401</b>  | <b>0.9423</b> | <b>0.9343</b> |
| XGBoost       | 0.9285        | 0.9400            | 0.9375         | 0.9387        | 0.9286        |
| MLP           | 0.9241        | 0.9355            | 0.9311         | 0.9329        | 0.9243        |
| Random Forest | 0.9233        | 0.9342            | 0.9314         | 0.9327        | 0.9234        |

### 结果分析：

SVM 在所有六项指标上均取得最优成绩。RBF 核通过将数据映射到高维空间, 在中小规模数据集上展现出优秀的分类能力。Macro F1 普遍高于 Weighted F1 (SVM: 0.9423 vs 0.9343), 说明模型在少数类 (BOMBAY 仅 3.8%) 上的表现并未显著拖累整体指标, 各类别分类质量较为均衡。

### 混淆矩阵分析 (以性能最优的 SVM 为例)：

- DERMASON (634 样本): 591 正确, 主要误判为 SIRA (35) 和 SEKER (8)
- **BOMBAY (102 样本): 100 正确, 仅 2 例误判——分类近乎完美**
- BARBUNYA (250 样本): 233 正确, 13 例误判为 CALI
- SIRA (459 样本): 413 正确, 35 例误判为 DERMASON (双向混淆)

BOMBAY 虽样本最少, 但分类准确率极高 (98%), 说明其特征与其余类别有明确的决策边界。DERMASON SIRA 的双向混淆说明这两类干豆在几何形态上高度相似。

四种算法的完整混淆矩阵热力图见 `output/figures/cm_*.png`。

5.3 过拟合分析

| 算法             | 训练集准确率        | 测试集准确率        | 差异（过拟合程度）             |
|----------------|---------------|---------------|-----------------------|
| SVM            | 0.9378        | 0.9342        | 0.0036（几乎无过拟合）        |
| MLP            | 0.9256        | 0.9241        | 0.0015（几乎无过拟合）        |
| Random Forest  | 0.9913        | 0.9233        | 0.0680（明显过拟合）         |
| <b>XGBoost</b> | <b>0.9999</b> | <b>0.9285</b> | <b>0.0714（最严重过拟合）</b> |

分析：

树模型（RF 和 XGBoost）展现了强大的数据拟合能力——它们在训练集上几乎达到完美精度，但测试集精度反而低于 SVM。这揭示了集成树模型的一个典型特征：强大的表达能力需要配合适当的正则化。虽然本实验中 XGBoost 的最大深度仅设为 6，但 200 轮迭代仍然导致了轻微过拟合。

相比之下，SVM 的结构风险最小化原则天然具备良好的泛化能力，MLP 通过早停 (early\_stopping) 和自适应学习率有效抑制了过拟合。SVM 的训练-测试差异仅 0.36 个百分点。

过拟合对比柱状图见 output/figures/overfitting\_analysis.png。

5.4 Loss 曲线分析（迭代型算法）

**XGBoost（梯度提升）**：训练损失 (mlogloss) 从初始约 1.2 单调下降至接近 0。验证损失在前 50 轮快速下降，约 100 轮后趋于平稳，未出现明显上升——说明模型在验证集上未过拟合到损失反弹的程度，早停机制有效地在验证损失收敛时终止了训练。

**MLP（神经网络）**：损失曲线呈现典型的阶梯式下降形态。训练约 40 个 epoch 后损失下降放缓，触发自适应学习率衰减 (adaptive learning rate)，随后损失继续阶梯式下降至收敛。验证分数随 epoch 波动上升，最终稳定在 0.92 附近，表明模型已充分学习。

Loss 曲线对比图见 output/figures/loss\_curves.png。

5.5 推理速度对比

| 算法            | 推理时间 (ms/样本)  | 相对速度   | 适用场景      |
|---------------|---------------|--------|-----------|
| <b>MLP</b>    | <b>0.0006</b> | 1×（最快） | 实时推理、边缘设备 |
| XGBoost       | 0.0030        | 5×     | 实时批量预测    |
| Random Forest | 0.0097        | 16×    | 常规服务      |
| SVM           | 0.0473        | 79×    | 离线高精度分析   |

分析：

- MLP 推理最快，因为前向传播仅需三层矩阵乘法和 ReLU 激活，现代 CPU 的 BLAS 库对此高度优化。
- XGBoost 的推理速度也非常优秀 (0.003 ms/样本)，得益于其树结构的 C++ 底层实现。
- SVM 最慢——RBF 核预测时需要计算测试样本与每一个支持向量 (SV) 的核函数值，其复杂度为  $O(n_{SV} \times d)$ ，而本实验 SVM 保留了大量支持向量。

## 5.6 鲁棒性对比

向测试集特征注入两种不同类型的噪声，观察模型精度变化：

(1) 高斯噪声 (Gaussian Noise) —向特征添加  $N(0, \text{std} \times \text{level})$  的随机扰动：

| 噪声强度 | SVM           | XGBoost | MLP    | Random Forest |
|------|---------------|---------|--------|---------------|
| 0.00 | 0.9342        | 0.9285  | 0.9241 | 0.9233        |
| 0.01 | 0.9342        | 0.9298  | 0.9237 | 0.9217        |
| 0.05 | <b>0.9354</b> | 0.9294  | 0.9233 | 0.9269        |
| 0.10 | 0.9314        | 0.9233  | 0.9197 | 0.9229        |

(2) 椒盐噪声 (Salt-and-Pepper Noise) —随机将 level 比例的像素替换为特征极值：

| 噪声强度 | SVM    | XGBoost | MLP    | Random Forest |
|------|--------|---------|--------|---------------|
| 0.00 | 0.9342 | 0.9285  | 0.9241 | 0.9233        |
| 0.01 | 0.9088 | 0.9245  | 0.9003 | <b>0.9221</b> |
| 0.05 | 0.8010 | 0.9136  | 0.8070 | <b>0.9144</b> |
| 0.10 | 0.6964 | 0.8995  | 0.7202 | <b>0.9023</b> |

分析：

1. 椒盐噪声的危害远大于高斯噪声。在 0.10 强度的椒盐噪声下，SVM 精度下降 23.8 个百分点至 0.6964，MLP 下降 20.4 个百分点至 0.7202。而高斯噪声下最大下降仅约 0.6%。这是因为椒盐噪声将特征值替换为极值（最小或最大），完全破坏了样本的几何特征信息，而高斯噪声仅添加小幅扰动。
2. Random Forest 在椒盐噪声下表现最鲁棒（0.10 强度下仍保持 0.9023），这是因为树模型基于特征的分裂点做决策，个别特征的极端值不会影响所有树的分裂路径。
3. SVM 在高斯噪声下最鲁棒（下降仅 0.28%），但对椒盐噪声最敏感，因为 RBF 核依赖样本间的距离度量，极端值会严重扭曲距离计算。
4. 两种噪声类型的对比充分说明了鲁棒性评估必须考虑噪声的多样性——仅凭高斯噪声测试会得出“所有模型都很鲁棒”的错误结论。

鲁棒性对比曲线图（含高斯和椒盐噪声双图）见 `output/figures/robustness_curves.png`。

## 5.7 每类别 F1 对比

为了分析模型在不同类别上的表现差异，以下列出每个类别的 F1 分数：

| 类别            | SVM           | XGBoost       | MLP           | Random Forest |
|---------------|---------------|---------------|---------------|---------------|
| BARBUNYA      | 0.9376        | 0.9287        | 0.9155        | 0.9146        |
| <b>BOMBAY</b> | <b>0.9901</b> | <b>1.0000</b> | <b>0.9901</b> | <b>1.0000</b> |
| CALI          | 0.9365        | 0.9340        | 0.9270        | 0.9181        |



| 类别          | SVM           | XGBoost       | MLP           | Random Forest |
|-------------|---------------|---------------|---------------|---------------|
| DERMASON    | 0.9278        | 0.9188        | 0.9190        | 0.9187        |
| HOROZ       | 0.9564        | 0.9591        | 0.9590        | 0.9563        |
| SEKER       | 0.9541        | 0.9444        | 0.9415        | 0.9391        |
| <b>SIRA</b> | <b>0.8939</b> | <b>0.8858</b> | <b>0.8786</b> | <b>0.8822</b> |

分析：

- **BOMBAY** 几近完美 (F1 0.99)：尽管样本最少 (3.8%)，但其几何特征与其他类别有清晰的决策边界
- **SIRA** 系统性偏弱 (F1 0.88-0.89)：SIRA 与 DERMASON 在形状特征上高度重叠，导致双向误分类
- SVM 在 6/7 类别上取得最高 F1，但各类模型间的每类别排名基本一致

每类别 F1 对比柱状图见 `output/figures/per_class_f1.png`。

## 5.8 综合性能对比

| 评估维度        | 最优算法            | 次优算法                |
|-------------|-----------------|---------------------|
| 测试精度        | SVM (93.42%)    | XGBoost (92.85%)    |
| 泛化能力 (防过拟合) | MLP (差异 0.15%)  | SVM (差异 0.36%)      |
| 推理速度        | MLP (0.0006 ms) | XGBoost (0.0030 ms) |
| 噪声鲁棒性       | SVM (下降 0.28%)  | RF (下降 0.16%)       |

## 6. 系统展示 (评分占比 30%)

本节展示完整的工程架构、命令行调用方式及 GitHub 展示。项目代码已开源，教师可通过以下链接在线查看：

**GitHub 仓库：** <https://github.com/joyboy-type/machine-learning-final>

### 6.1 工程模块架构

项目采用模块化设计，严格遵循单一职责原则。每个 `.py` 文件仅负责一个明确的功能领域，通过 `run.py` 作为统一入口调度所有模块。

```
机器学习期末大作业 /
+-- config.py           # 全局路径配置、常量定义
+-- data_loader.py      # 数据加载接口（原始/清洗/处理后）
+-- analysis.py         # 数据分析：污染检测、统计描述、4张可视化图
+-- preprocess.py       # 数据预处理：5步清洗流水线 + 标准化 + 可选PCA
+-- train.py           # 模型注册中心：装饰器模式，4种算法的统一训练接口
+-- evaluate.py         # 全维度评估：精度/F1/混淆矩阵/鲁棒性/过拟合/推理速度
+-- utils.py           # 工具函数：高斯/椒盐噪声生成、推理计时
```

```

+-- run.py          # CLI入口：argparse命令行解析，一键式pipeline调度
+-- DryBeanDataset/ # 原始数据目录（3个CSV）
+-- output/
    +-- figures/    # 14张评估图表
    +-- results/    # JSON格式完整实验结果

```

设计原则：

- 单一职责：每个 .py 文件仅负责一个明确的功能领域
- 开闭原则：train.py 的 @register 装饰器机制使得添加新算法只需编写一个工厂函数，无需修改 run.py 或 evaluate.py
- Pipeline 自动化：run.py --all 一键运行从数据分析到评估图表的完整流程
- 命令行驱动：无 GUI，所有操作通过命令行参数控制，符合课程要求

## 6.2 命令行接口

系统通过 run.py 提供统一的命令行入口，支持灵活的参数组合。下图展示了命令行帮助界面：

```

Terminal: python run.py --help

$ python run.py --help
usage: run.py [-h] [--algorithm ALGORITHM] [--pca] [--all]
              [--analysis-only] [--preprocess-only]

Dry Bean Classification - ML Pipeline

optional arguments:
  --algorithm ALGORITHM  Algorithm(s), comma-separated.
                        Available: svm, rf, xgboost, mlp, all
  --pca                  Use PCA dimensionality reduction
  --all                  Full pipeline: analysis + preprocessing
                        + training + evaluation
  --analysis-only        Run only data analysis
  --preprocess-only      Run only preprocessing

Examples:
python run.py --algorithm svm
python run.py --algorithm all
python run.py --algorithm svm,xgboost
python run.py --all
python run.py --analysis-only
python run.py --preprocess-only

```

Figure 1: CLI Help

支持的命令：

```

python run.py --all          # 一键完整流程
python run.py --analysis-only # 仅数据分析
python run.py --preprocess-only # 仅预处理
python run.py --algorithm svm # 单算法训练
python run.py --algorithm svm,xgboost # 多算法训练
python run.py --algorithm all --pca # PCA 降维模式
python run.py --help         # 查看帮助

```

下图展示了运行 `python run.py --algorithm all` 的完整输出，包含数据加载、预处理报告、四种算法逐个训练与评估、以及最终汇总表：

```

Terminal: python run.py --algorithm all

$ python run.py --algorithm all

=====
STEP 2: DATA PREPROCESSING
=====
Raw data loaded: train=9527, test=2737, val=1347

--- Cleaning Report ---
train_rows_raw: 9527
train_bad_labels_removed: 0
test_rows_raw: 2737
test_bad_labels_removed: 0
total_before: 13611
total_after: 12284
total_removed: 1327

=====
STEP 3: MODEL TRAINING & EVALUATION
=====

Training: SVM
Training time: 0.79s | Train accuracy: 0.9378
Test accuracy: 0.9342 | F1 (macro): 0.9423

Training: XGBOOST
Training time: 1.62s | Train accuracy: 0.9999
Test accuracy: 0.9285 | F1 (macro): 0.9387

Training: RF
Training time: 0.52s | Train accuracy: 0.9913
Test accuracy: 0.9233 | F1 (macro): 0.9327

Training: MLP
Training time: 0.66s | Train accuracy: 0.9256
Test accuracy: 0.9241 | F1 (macro): 0.9329

=====
FINAL SUMMARY
=====
Algorithm  Accuracy  F1(Macro)  F1(Wtd)  Inf(ms)  Train(s)
-----
SVM          0.9342      0.9423     0.9343    0.0480    0.79
XGBOOST      0.9285      0.9387     0.9286    0.0028    1.62
MLP          0.9241      0.9329     0.9243    0.0006    0.66
RF           0.9233      0.9327     0.9234    0.0104    0.52

Done. All figures saved to output/figures/

```

Figure 2: CLI Run

从运行输出可以清晰看到：数据从 13,611 行清洗至 12,284 行；四种算法依次训练（SVM 0.79s, RF 0.52s, XGBoost 1.62s, MLP 0.66s）；最后打印精度汇总表。整个流程无需任何人工交互，完全自动化。

### 6.3 模型注册与扩展机制

`train.py` 使用装饰器模式实现算法注册。添加新算法仅需编写一个工厂函数并用 `@register` 装饰：

```

@register("catboost")
def build_catboost():
    return CatBoostClassifier(iterations=200, depth=6), True

```

系统会自动将新算法纳入 `--algorithm all` 的运行列表，无需修改 `run.py` 或 `evaluate.py` 的任何代码。这一设计体现了开闭原则（对扩展开放，对修改关闭）。

### 6.4 GitHub 展示

项目已按工程标准上传至 GitHub，仓库地址：<https://github.com/joyboy-type/machine-learning-final>。

仓库包含内容：

```
GITHUB REPOSITORY
https://github.com/joyboy-type/machine-learning-final

machine-learning-final/
+-- README.md          # Project overview, data description,
|                       # algorithms, results summary, usage
+-- requirements.txt    # Python dependencies
+-- .gitignore          # Exclude dataset, outputs, cache
+-- config.py           # Global paths and constants
+-- data_loader.py      # Raw/clean/processed data loading
+-- analysis.py         # Data quality analysis (4 charts)
+-- preprocess.py       # 5-step cleaning + standardization + PCA
+-- train.py            # Model registry (@register pattern)
+-- evaluate.py         # 5-dimension evaluation, 7 chart types
+-- utils.py            # Noise generation, inference timing
+-- run.py              # CLI entry point (argparse)
+-- data/               # Dataset directory (CSV files)
+-- output/             # Generated results (auto-created)
    +-- figures/         # 14 evaluation charts
    +-- results/         # JSON experiment data
```

Figure 3: GitHub Repo

| 内容   | 文件                 | 说明                                                        |
|------|--------------------|-----------------------------------------------------------|
| 项目说明 | README.md          | 数据描述、污染详情（4 类）、处理流程（5 步）、算法列表、完整精度表、快速开始指南                |
| 依赖清单 | requirements.txt   | numpy, pandas, scikit-learn, matplotlib, seaborn, xgboost |
| 忽略规则 | .gitignore         | 排除数据集文件、生成输出、Python 缓存、IDE 配置                             |
| 模块源码 | 8 个 .py 文件         | 完整模块化代码，每文件单一职责                                           |
| 论文报告 | .md + .pdf + .docx | 三种格式的完整项目论文                                               |

README 展示的数据描述内容包括：

- 数据集基本信息（16 特征、7 类别、13,611 样本）
- 数据污染详情（Leet-speak、大小写、空格、缺失值“?”和 NaN、“cm”后缀、重复行）
- 类别分布表（含不均衡提示）
- 数据处理 5 步流水线
- 4 种算法及其超参数
- 完整测试精度表（Accuracy、F1 Macro、F1 Weighted、Precision、Recall）
- 过拟合分析表、推理速度表、双噪声鲁棒性表

6.5 实验结果汇总

所有算法的测试集精度汇总如下（更详细的结果见第五章和第 5.7 节每类别分析）：

| Algorithm        | Accuracy      | F1<br>(Macro) | F1<br>(Weighted) | Inference<br>(ms) | Training<br>(s) |
|------------------|---------------|---------------|------------------|-------------------|-----------------|
| SVM<br>(RBF)     | <b>0.9342</b> | <b>0.9423</b> | <b>0.9343</b>    | 0.0480            | 0.79            |
| XGBoost          | 0.9285        | 0.9387        | 0.9286           | 0.0028            | 1.62            |
| MLP              | 0.9241        | 0.9329        | 0.9243           | 0.0006            | 0.66            |
| Random<br>Forest | 0.9233        | 0.9327        | 0.9234           | 0.0104            | 0.52            |

## 7. 综合讨论

### 7.1 算法选型建议

根据不同应用场景的需求，推荐以下选型策略：

| 场景        | 推荐算法          | 理由                |
|-----------|---------------|-------------------|
| 精度优先（如质检） | SVM (RBF)     | 93.42% 最高精度，泛化最好  |
| 实时推理（如产线） | MLP           | 0.0006 ms/样本，最快推理 |
| 平衡方案（推荐）  | XGBoost       | 精度接近 SVM，速度快 15 倍 |
| 可解释性要求    | Random Forest | 特征重要性天然可解释        |

### 7.2 特征工程反思

相关性分析显示多个特征高度相关（Area ConvexArea 相关系数  $\sim 1.0$ ），表明 16 个几何特征存在信息冗余。这是从图像中提取几何描述子的典型特征——面积、周长、凸包面积等存在数学上的内在关联。PCA 降维或基于互信息的特征选择可能在不损失精度的情况下将特征压缩至 8-10 维，这有待进一步实验验证。

### 7.3 数据清洗的价值

本次数据清洗共移除了 9.75% 的数据。如果没有这些清洗步骤：

- 类别数从 7 膨胀到 25，模型将学习错误的类别边界
- 缺失值导致 `fit()` 抛出异常，训练直接失败
- “cm”后缀和“?”占位符导致列被读为 object 类型，无法进行数值计算
- 49 个重复样本可能造成过拟合评估偏差

人工污染 + 清洗的流程，有效模拟了真实工业场景中的数据预处理全流程。

## 8. 结论

本项目完成了 Dry Bean Dataset 的完整机器学习工程流程：

1. **数据质量分析** (5%) 揭示了三类污染：标签错误 (25→7 类, 含 Leet-speak/大小写/空格)、缺失值 (NaN + “?”共 1,065 个)、格式错误 (“cm”后缀 388 个)，并生成了 4 张分析可视化图。

2. **数据处理 (30%)** 实现了 5 步清洗流水线 (标签映射 → 类型修复 → 缺失删除 → 去重 → IQR 裁剪) + Z-score 标准化, 数据从 13,611 行清洗至 12,284 行 (保留率 90.25%), 支持可选的 PCA 降维。
3. **多算法实验 (30%)** 对比了 4 种算法 (含课外自学的 XGBoost) 在 5 个维度的表现:
  - SVM 精度最高 (93.42%), 鲁棒性最好
  - XGBoost 综合最优 (92.85%, 速度精度均衡)
  - MLP 推理最快 (0.0006 ms/样本)
  - 生成 12 张评估图表和完整 JSON 结果
4. **系统展示 (30%)** 提供了模块化工程架构、命令行一键调用、GitHub 完整展示。

#### 未来改进方向

- 引入 SMOTE 过采样或类别加权损失函数, 缓解 BOMBAY 样本不足问题
- 扩充算法对比范围: CatBoost、LightGBM、朴素贝叶斯
- 使用 Optuna 进行自动化超参数搜索
- 探索构造交互特征 (如 Area/Perimeter 作为形状复杂度指标)

## 9. 课程总结 (评分占比 5%)

### 9.1 课程收获

通过本学期的《机器学习与项目实践》课程, 我系统地建立了机器学习从理论到实践的完整知识体系:

#### 理论层面:

- 掌握了线性回归、逻辑回归、KNN、SVM、人工神经网络 (ANN) 等经典算法的数学原理和推导过程
- 理解了偏差-方差权衡、过拟合/欠拟合、正则化、核方法等核心概念
- 通过实验作业深入理解了每个算法的超参数含义和调参策略

#### 实践层面:

- 从零搭建了完整的机器学习工程 Pipeline (数据加载 → 清洗 → 特征工程 → 训练 → 评估)
- 掌握了 scikit-learn、XGBoost 等主流框架的使用
- 学会了模块化代码设计、命令行工具构建、GitHub 项目展示等软件工程实践
- 通过 Dry Bean 期末项目, 体验了从“脏数据”到“可部署模型”的真实工业流程

#### 项目能力:

- 学会了如何系统化地分析数据质量问题 (标签错误、缺失值、格式异常、异常值)
- 掌握了多维度模型对比的方法论 (精度、速度、鲁棒性、泛化能力)
- 形成了“假设 → 实验 → 分析 → 结论”的科学研究思维

### 9.2 课程评价

#### 课程优点:

1. **理论与实践紧密结合:** 每周实验作业配合课堂讲授, 当天完成实验确保了知识的即时巩固。实验内容从线性回归逐步推进到 ANN, 难度梯度合理。

- 2. “脏数据”设计巧妙：期末数据集的污染设计（Leet-speak、格式错误等）高度模拟了真实世界的数  
据质量问题，这种”先污染后治理”的方式比直接使用干净数据更有教学价值。
- 3. 工程化导向明确：课程不仅关注算法精度，还强调代码架构、命令行接口、GitHub 展示等工程素  
养，符合业界实际需求。
- 4. 课堂互动充分：当堂完成作业并检查的机制保证了学习效率，附加题的设置激励了深度探索。

改进建议：

- 1. 可引入更多关于深度学习的内容（如 CNN 在图像分类中的应用，与本课程的图像特征数据集形成  
呼应）
- 2. 建议增加一次关于模型部署的专题（如 Flask API 封装或 ONNX 模型导出）
- 3. 期末项目可增加同学互评环节，促进不同思路的交流碰撞
- 4. 可考虑引入 Kaggle 竞赛机制，增加课程的趣味性和挑战性

9.3 自我反思

通过本课程，我最大的收获是认识到：机器学习中 80% 的工作在于数据理解 and 处理，而不仅仅是模型调参。本次期末项目中，数据清洗和特征工程阶段占据了大量时间——识别污染类型、设计修复策略、验证清洗效果。这段经历让我深刻理解了工业界”数据和特征决定了机器学习的上限，而模型和算法只是逼近这个上限”这一经典论断。

附录

A. 实验环境

| 项目           | 版本/信息                            |
|--------------|----------------------------------|
| 操作系统         | macOS 26 (ARM64 / Apple Silicon) |
| Python       | 3.9.6                            |
| scikit-learn | 1.x                              |
| XGBoost      | 2.1.4                            |
| NumPy        | 1.x                              |
| Pandas       | 1.x                              |
| 可视化          | Matplotlib 3.x + Seaborn 0.13    |
| PDF 生成       | Pandoc 3.9 + Tectonic            |

B. 图表索引

| 编号  | 内容                       | 文件路径                                        |
|-----|--------------------------|---------------------------------------------|
| 1   | 类别分布柱状图                  | output/figures/01_class_distribution.png    |
| 2   | 特征箱线图                    | output/figures/02_feature_boxplot.png       |
| 3   | 特征相关性热力图                 | output/figures/03_correlation_heatmap.png   |
| 4   | 按类特征分布直方图                | output/figures/04_feature_hist_by_class.png |
| 5-8 | 混淆矩阵<br>(SVM/RF/XGB/MLP) | output/figures/cm_*.png                     |
| 9   | 模型性能指标对比                 | output/figures/metrics_comparison.png       |

| 编号 | 内容            | 文件路径                                    |
|----|---------------|-----------------------------------------|
| 10 | 高斯噪声鲁棒性曲线     | output/figures/robustness_curves.png    |
| 11 | 椒盐噪声鲁棒性曲线     | output/figures/robustness_curves.png    |
| 12 | 训练/验证 Loss 曲线 | output/figures/loss_curves.png          |
| 13 | 过拟合分析         | output/figures/overfitting_analysis.png |
| 14 | 每类别 F1 对比     | output/figures/per_class_f1.png         |

### C. 项目代码链接

- **GitHub 文件夹:** GitHub/ (包含完整源码、README、requirements.txt、.gitignore)
- **作业提交文件夹:** 作业提交/ (包含源码 + 数据集 + 所有生成结果)

本报告由 *Joyboy* 完成, 作为 *2026 AIT209* 机器学习与项目实践课程的期末考核作业。